作业 6

1.

（1）

- 按稳定流计算，可用抽水井数据与观测孔数据计算参数，也可用两个观测孔的数据计算参数，它们都有如 Thiem 公式的形式：

$$K = \frac{Q}{2\pi M(H_2 - H_1)} \ln \frac{r_2}{r_1} = \frac{Q}{2\pi M(s_1 - s_2)} \ln \frac{r_2}{r_1}$$

```python
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)

Q = 67.2
M = 18.5
rw = 0.1015
r1 = 2.0
r2 = 25.0
hw = 20.65
h1 = 21.12
h2 = 22.05

K = Q / M / (hw - h1) /2/ np.pi * np.log(rw / r1)
print("抽水井与观测孔 1: ")
print(f' 渗透系数 K = {K:.4f} m^2/d')
```

```

K = Q / M / (hw - h2) /2/ np.pi * np.log(rw / r2)
print("抽水井与观测孔 2: ")
print(f' 渗透系数 K = {K:.4f} m^2/d')

K = Q / M / (h1 - h2) /2/ np.pi * np.log(r1 / r2)
print("观测孔 1、2: ")
print(f' 渗透系数 K = {K:.4f} m^2/d')
```

抽水井与观测孔1:
渗透系数 K = 3.6666 m^2/d
抽水井与观测孔2:
渗透系数 K = 2.2739 m^2/d
观测孔1、2:
渗透系数 K = 1.5701 m^2/d

- 不用观测孔，用 Dupuit 公式与影响半径经验公式计算参数：

$$K = \frac{Q}{2\pi M s_w} \ln \frac{R}{r_w}, R = 10 s_w \sqrt{K}$$

```
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)

def calc_para(Q, M, sw, rw, tolerance=1e-4):
    def calc( Q, M, sw, rw, tolerance=1e-4):
        K = 1.0  # 初始渗透系数
        error = np.inf  # 初始误差设为无穷大
        while error > tolerance:
            R = 10 * sw * np.sqrt(K)  # 根据经验公式计算 R
            new_K = Q / (2 * np.pi * M * sw) * np.log(R / rw)  # 根据 Dupuit 公式计算新的
            error = np.abs(new_K - K)  # 计算绝对误差
            K = new_K  # 更新 K 值
```

```
        return K, R

    vec_func = np.vectorize(calc)
    return vec_func(Q, M, sw, rw, tolerance=1e-4)

Q = 67.2
M = 18.5
rw = 0.1015
sw = 23-20.65

# 调用函数计算渗透系数和影响半径
K, R = calc_para(Q, M, sw, rw)

print(f'渗透系数 K = {K:.4f} m^2/d')
print(f'影响半径 R = {R:.4f} m')
```

```
渗透系数 K = 1.3790 m^2/d
影响半径 R = 27.5956 m
```

- 也可按非稳定流计算，采用抽水井与一个观测孔的数据计算参数，或者两个观测孔数据计算参数，与稳定流结果一样。由于没有时间 t，所以求不出贮水系数。

（2）若考虑有井损，试对参数计算结果进行评估。

用抽水井与观测孔数据没有考虑井损，精度受影响。而两个观测孔的 thiem 公式，不受井损影响，相对来水较为可靠。

（3）若初始水位为 23 m，计算本次抽水的影响半径。

$$\lg R = \frac{s_1 \lg r_2 - s_2 \lg r_1}{s_1 - s_2}$$

```
import numpy as np

# 设置显示选项
```

```
np.set_printoptions(precision=4)

Q = 67.2
M = 18.5
rw = 0.1015
r1 = 2.0
r2 = 25.0
sw = 23 - 20.65
s1 = 23 - 21.12
s2 = 23 - 22.05

logR = (s1 * np.log10(r2) - s2 * np.log10(r1)) / (s1 - s2)
R = 10**logR
print(f' 影响半径 R = {R:.4f} m')
```

影响半径 R = 329.9434 m

（4）

按 Dupuit 公式验证：

```
import numpy as np

Q = 67.2
M = 18.5
rw = 0.1015
r1 = 2.0
r2 = 25.0
sw = 23 - 20.65
s1 = 23 - 21.12
s2 = 23 - 22.05

print(' 观测值:')
print(f'sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
print(' 计算值:')
```

```
15
16  R = 329.9434
17  print(f' 影响半径 R = {R:.4f} m')
18  K = 3.6666
19  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
20  s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
21  s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
22  print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
23  K = 2.2739
24  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
25  s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
26  s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
27  print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
28  K = 1.5701
29  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
30  s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
31  s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
32  print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
33  K = 1.3790
34  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
35  s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
36  s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
37  print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
38
39  R = 27.5956
40  print(f' 影响半径 R = {R:.4f} m')
41  K = 3.6666
42  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
43  s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
44  s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
45  print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
46  K = 2.2739
47  sw = Q / M / K / 2 / np.pi * np.log(R / rw)
```

```
s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
K = 1.5701
sw = Q / M / K / 2 / np.pi * np.log(R / rw)
s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
K = 1.3790
sw = Q / M / K / 2 / np.pi * np.log(R / rw)
s1 = Q / M / K / 2 / np.pi * np.log(R / r1)
s2 = Q / M / K / 2 / np.pi * np.log(R / r2)
print(f'K = {K:.4f} m^2/d, sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
```

观测值:

```
sw = 2.3500, s1 = 1.8800, s2 = 0.9500 m
```

计算值:

```
影响半径 R = 329.9434 m
K = 3.6666 m^2/d, sw = 1.2750, s1 = 0.8050, s2 = 0.4068 m
K = 2.2739 m^2/d, sw = 2.0560, s1 = 1.2981, s2 = 0.6560 m
K = 1.5701 m^2/d, sw = 2.9775, s1 = 1.8800, s2 = 0.9500 m
K = 1.3790 m^2/d, sw = 3.3902, s1 = 2.1405, s2 = 1.0816 m
影响半径 R = 27.5956 m
K = 3.6666 m^2/d, sw = 0.8838, s1 = 0.4138, s2 = 0.0156 m
K = 2.2739 m^2/d, sw = 1.4251, s1 = 0.6673, s2 = 0.0251 m
K = 1.5701 m^2/d, sw = 2.0639, s1 = 0.9664, s2 = 0.0364 m
K = 1.3790 m^2/d, sw = 2.3499, s1 = 1.1003, s2 = 0.0414 m
```

从计算结果可以看出，由于有井损影响，抽水井降深误差较大，按 Dupuit 公式验证，用 Thiem 公式计算的渗透系数与影响半径较为可靠。

按 Hantush-Jacob 稳态解计算参数：

$$K = \frac{0.366Q}{M(s_1 - s_2)} \lg \frac{r_2}{r_1}, \quad B = 0.89\, r_1 \times 10^{\frac{s_1 T}{0.366Q}}$$

```python
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)

Q = 67.2
M = 18.5
rw = 0.1015
r1 = 2.0
r2 = 25.0
sw = 23 - 20.65
s1 = 23 - 21.12
s2 = 23 - 22.05

K = 0.366 * Q / M / (s1 - s2) * np.log10(r2 / r1)
B = 0.89 * r1 * 10**(s1 * K * M / (0.366 * Q))
print(f'渗透系数 K = {K:.4f} m^2/d')
print(f'越流因素 B = {B:.4f} m')
```

```
渗透系数 K = 1.5681 m^2/d
越流因素 B = 293.6497 m
```

按 Hantush-Jacob 稳态解验证：

取 K = 1.5681 m^2/d, B = 293.6497 m

```python
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)

Q = 67.2
M = 18.5
```

```
rw = 0.1015
r1 = 2.0
r2 = 25.0
sw = 23 - 20.65
s1 = 23 - 21.12
s2 = 23 - 22.05
K = 1.5681
B = 293.6497

print(' 观测值:')
print(f'sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
print(' 计算值:')

sw = Q / M / K / 2 / np.pi * np.log(1.123 * B / rw)
s1 = Q / M / K / 2 / np.pi * np.log(1.123 * B / r1)
s2 = Q / M / K / 2 / np.pi * np.log(1.123 * B / r2)
print(f'sw = {sw:.4f}, s1 = {s1:.4f}, s2 = {s2:.4f} m')
```

观测值:
sw = 2.3500, s1 = 1.8800, s2 = 0.9500 m
计算值:
sw = 2.9811, s1 = 1.8822, s2 = 0.9510 m

由此可见，假设有越流对计算结果有影响，但是影响较小。

2.

根据 1 中的结果，用观测孔计算的参数相对可靠，

$$K = \frac{0.732Q}{(2H_0 - s_1 - s_2)(s_1 - s_2)} \lg \frac{r_2}{r_1}$$

```
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)
```

```

Q = 66.48
M = 10.0
rw = 0.1015
r1 = 2.1
r2 = 6.1
sw = 10 - 6.40
s1 = 10 - 8.68
s2 = 10 - 9.21

K = 0.732 * Q / (2 * 10 - s1 - s2) / (s1 - s2) * np.log10(r2 / r1)
print("观测孔 1、2: ")
print(f' 渗透系数 K = {K:.4f} m^2/d')
```

观测孔1、2:
渗透系数 K = 2.3768 m^2/d

3.

不考虑井损，以 Jacob 阶梯流量公式计算：

$$s = \frac{0.183Q_1}{T}\lg\frac{2.25Tt}{r_w^2S} + \frac{0.183(Q_2-Q_1)}{T}\lg\frac{2.25T(t-t_1)}{r_w^2S}$$

式中，$T = 18.3m^2/d, S = 10^{-4}, s_w = 0.20m, Q_1 = 100m^3/d, Q_2 = 200m^3/d,$ $t = 12h, t_1 = 6h$。

将时间单位换成 d，$t = 0.5d, t_1 = 0.25d$，带入式中，有

```
import numpy as np

# 设置显示选项
np.set_printoptions(precision=4)

Q1 = 100
Q2 = 200
```

```
T = 18.3
S = 0.0001
rw = 0.2
t = 0.5
t1 = 0.25

s = 0.183*Q1/T*np.log10(2.25*T*t/rw**2/S)+0.183*(Q2-Q1)/T*np.log10(2.25*T*(t-t1)/rw**2/

print(f"12h 后降深 s = {s:.4f} m")
```

```
12h后降深 s = 13.1221 m
```

4.

用 Jacob 公式计算：

$$s_1 = \frac{0.183Q}{T} \lg \frac{2.25Tt_1}{r_1^2 S}$$

$$s_2 = \frac{0.183Q}{T} \lg \frac{2.25Tt_2}{r_2^2 S}$$

令 $s_1 = s_2$, 两式相减，有

$$\frac{t_1}{r_1^2} = \frac{t_2}{r_2^2}$$

可求出 $t_2 = 18h$

```
import numpy as np

t1 = 2
r1 = 50
r2 = 150
t2 = t1*r2**2/r1**2
```

```

print(f" {t2:.0f} ")
```

```
 18
```

5.

做单位变换: $Q = 0.348 m^3/min$,

用 Jacob 公式计算：

$$s_1 = \frac{0.183Q}{T} \lg \frac{2.25Tt_1}{r^2S}$$

$$s_2 = \frac{0.183Q}{T} \lg \frac{2.25Tt_2}{r^2S}$$

两式相减，有

$$T = 0.183Q \frac{t_1 - t_2}{s_1 - s_2}$$

计算得

导水系数 `T = 0.0767 m^2/min`

带入第一式或第二式，求得

贮水系数 `S = 0.0031`

```
import numpy as np
Q = 0.348
r = 10
t1 = 10
t2 = 50
s1 = 0.62
s2 = 1.20

```

```
T = 0.183 * Q *np.log10(t1/t2) / (s1 - s2)
print(f"导水系数 T = {T:.4f} m^2/min")

S = 2.25*T*t1/r**2 * 10 **(-s1*T/0.183/Q)
print(f"贮水系数 S = {S:.4f} ")
```

```
导水系数 T = 0.0767 m^2/min
贮水系数 S = 0.0031
```